

APEXPLANET SOFTWARE PVT. LTD.

Cybersecurity & Ethical Hacking Internship

TASK 3 — WEB APPLICATION SECURITY TESTING REPORT

SQL Injection, Cross-Site Scripting (XSS) & CSRF

1. Objective

The objective of Task 3 was to perform controlled web application security testing on the Damn Vulnerable Web Application (DVWA) and demonstrate common web application vulnerabilities. The testing focused on SQL Injection, Stored Cross-Site Scripting (XSS), Reflected XSS, and Cross-Site Request Forgery (CSRF). The vulnerabilities were reproduced in the local lab environment and documented with screenshots, observations, and appropriate mitigation techniques.

2. Lab Environment

Component	Details
Operating System	Kali Linux
Target Application	DVWA (Damn Vulnerable Web Application)
Web Server	Apache
Target URL	http://127.0.0.1/DVWA/
Browser	Web Browser / Firefox
Environment	Local controlled security testing lab

3. Tools Used

The following tools and components were used during the assessment:

- Kali Linux
- DVWA
- Apache Web Server
- Web Browser / Firefox
- MySQL database

4. Testing Methodology

The assessment was performed against the intentionally vulnerable DVWA application running locally on Kali Linux. Each vulnerability was tested separately using controlled input and the resulting application behavior was observed. Screenshots were captured as evidence of successful

exploitation. After each test, suitable mitigation measures were identified.

Stage	Activity
1	Access and verify DVWA application
2	Test SQL Injection using normal and malicious input
3	Test Stored XSS and Reflected XSS
4	Test CSRF using a crafted state-changing request
5	Record results and capture screenshots
6	Document security impact and mitigation

5. SQL Injection

5.1 Testing

SQL Injection was tested through the DVWA SQL Injection functionality. A normal input of 1 was first submitted to establish a baseline response. A malicious input of 1' OR '1'='1 was then supplied to alter the SQL query logic and return multiple database records.

5.2 Database Information Exposure

A UNION-based SQL Injection payload was also tested to retrieve information from the users table. The payload used was: 1' UNION SELECT user,password FROM users -- . The

application returned usernames and password hashes, demonstrating that SQL Injection could expose sensitive database information.

5.3 Result and Observation

The SQL Injection tests were successful. The application accepted unsanitized input and returned additional database records. The UNION test further demonstrated exposure of usernames and password hashes. This indicates a high-risk SQL Injection vulnerability in the tested functionality.

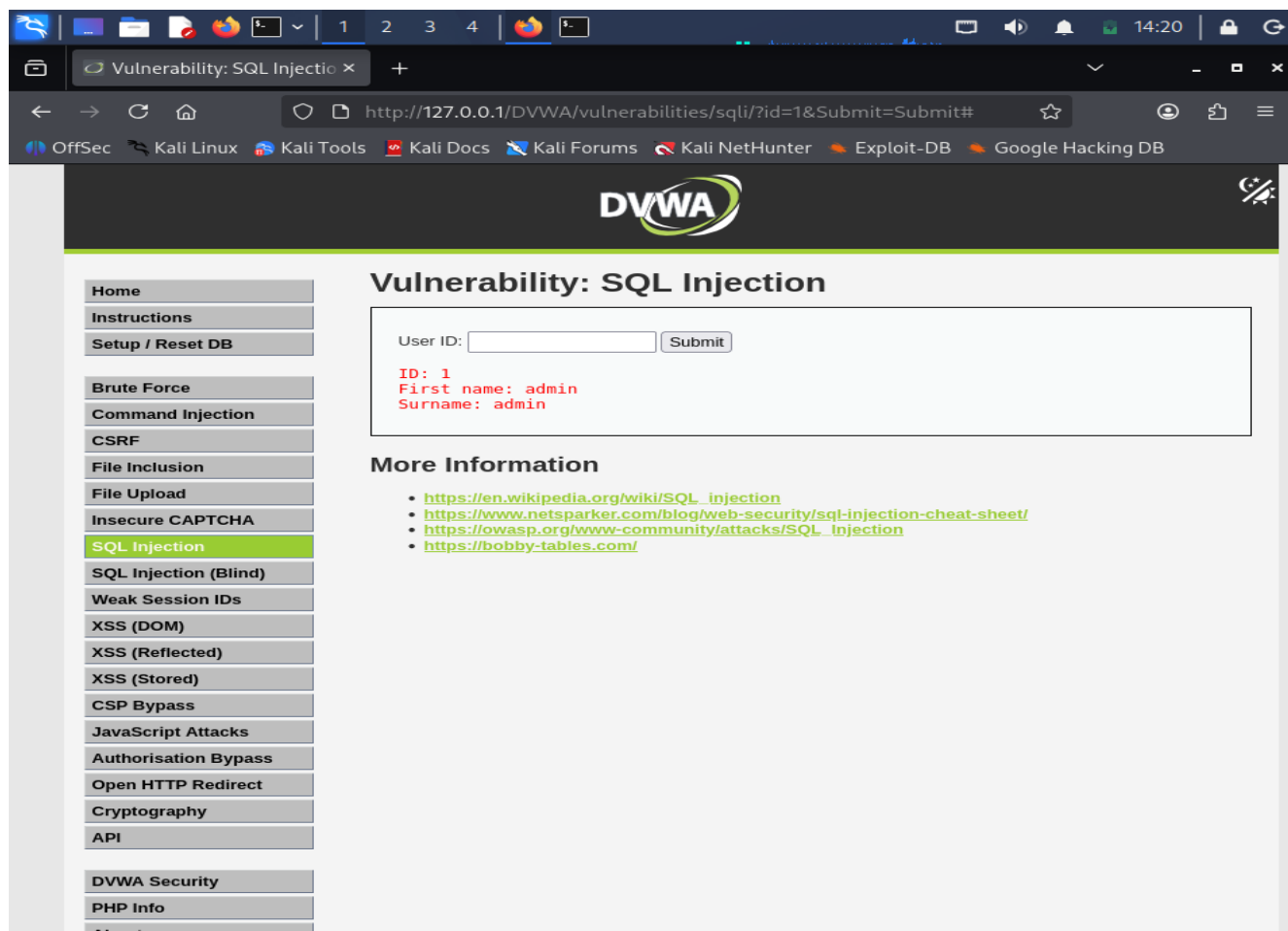


Figure 1: DVWA SQL Injection – Baseline Result

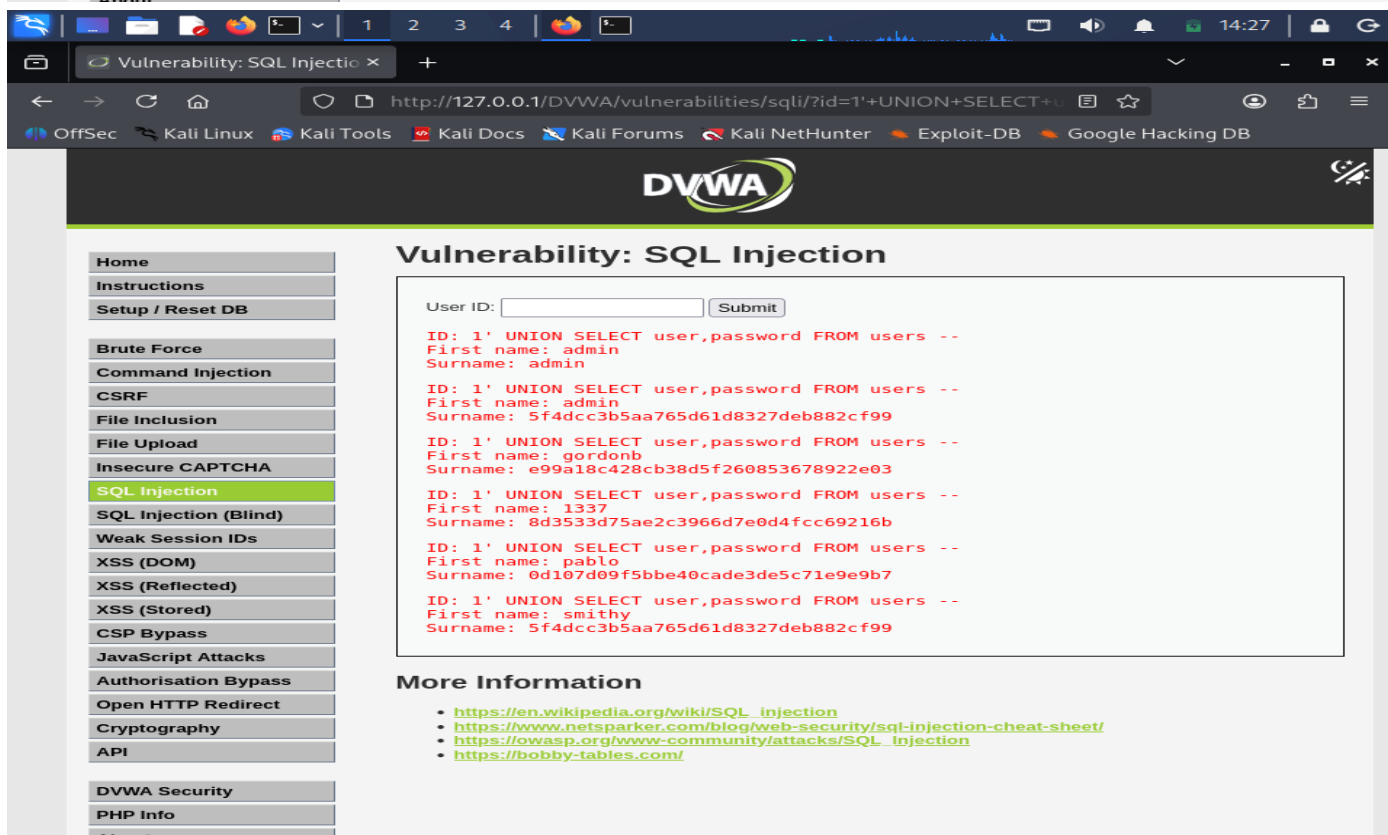


Figure 2a: DVWA SQL Injection – Multiple Records

5.4 Mitigation

SQL Injection can be mitigated by using prepared statements or parameterized queries instead of directly concatenating user input into SQL statements. Server-side input validation should also be applied, and database accounts should follow the principle of least privilege.

6. Stored Cross-Site Scripting (XSS)

6.1 Testing

Stored XSS was tested by submitting the payload `<script>alert('XSS')</script>` through the DVWA input field. The application stored the supplied input and later rendered it back to the page.

6.2 Result and Observation

The injected JavaScript executed in the browser when the stored content was displayed. This demonstrated that attacker-controlled script content could be stored by the application and executed in the context of users viewing the affected page.

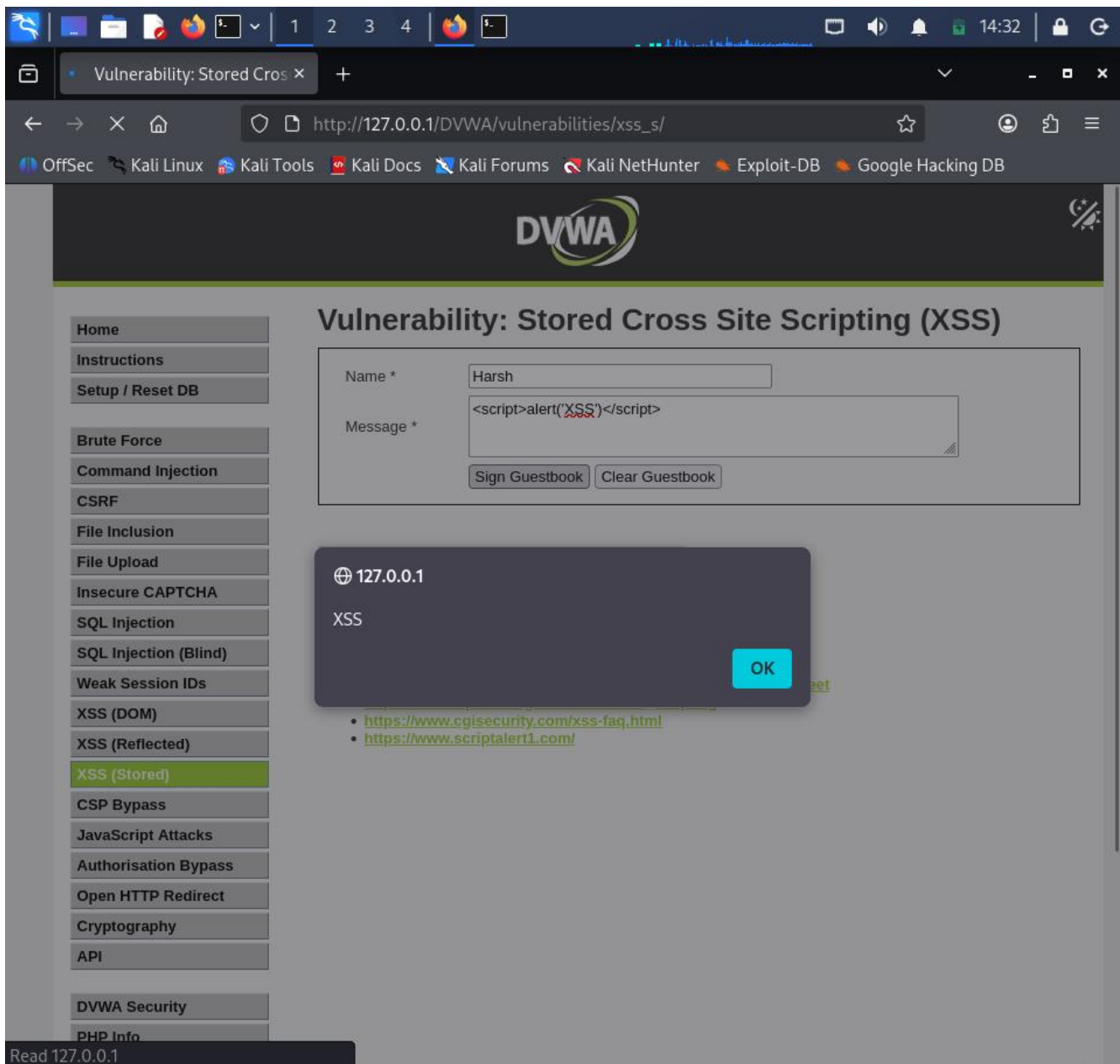


Figure 3: DVWA Stored XSS Execution

6.3 Mitigation

Stored XSS can be reduced through strict input validation and sanitization, context-appropriate output encoding, and the use of a suitable Content Security Policy (CSP). User-supplied content should not be rendered as executable HTML or JavaScript.

7. Reflected Cross-Site Scripting (XSS)

7.1 Testing

Reflected XSS was tested by supplying JavaScript through the application's request parameter. The payload `<script>alert('XSS')</script>` was used to verify whether the supplied input was reflected into the response without appropriate encoding.

7.2 Result and Observation

The payload was reflected by the application and executed in the browser, confirming a reflected XSS vulnerability. Unlike stored XSS, the malicious input was returned as part of the immediate response.

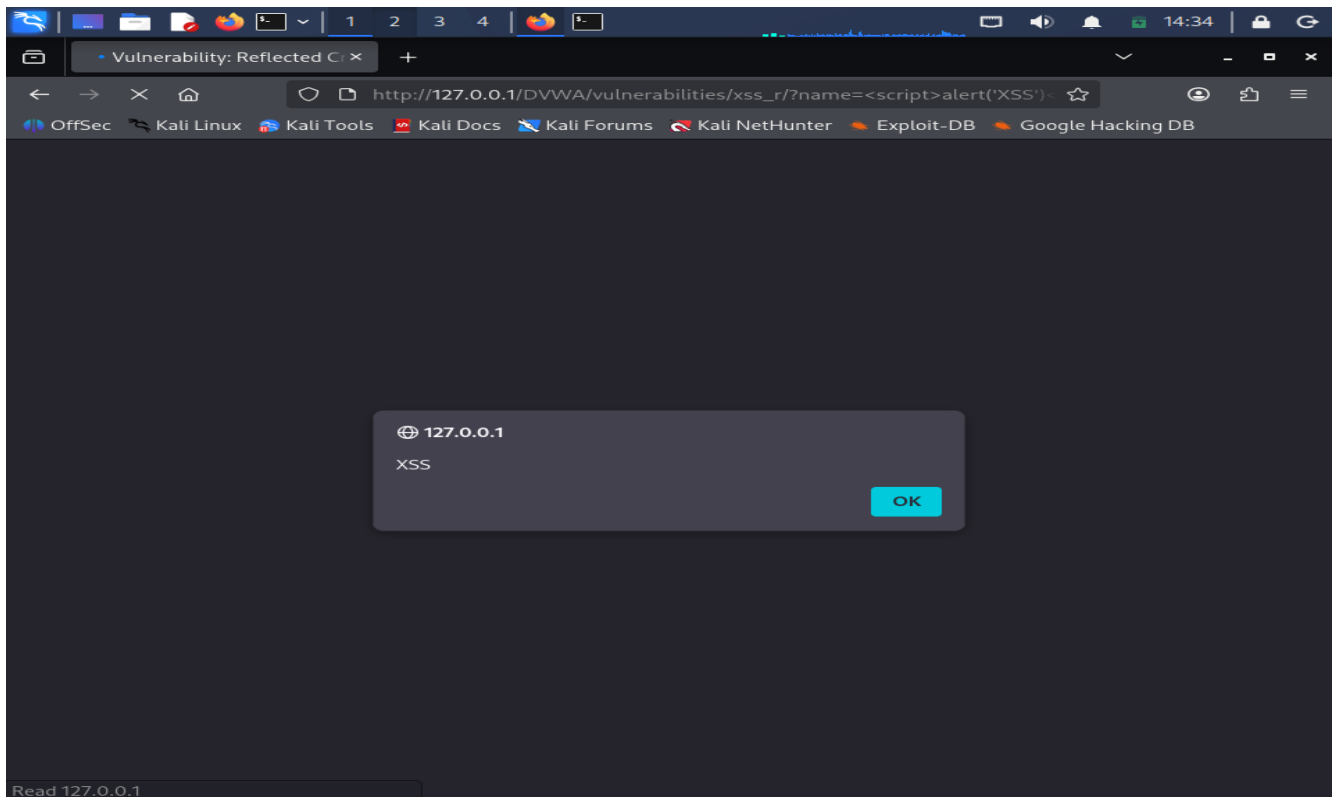


Figure 4: DVWA Reflected XSS Execution

7.3 Mitigation

Reflected XSS should be mitigated using server-side input validation and context-appropriate output encoding.

Applications should ensure that request parameters are not inserted directly into HTML, JavaScript, or other executable contexts.

8. Cross-Site Request Forgery (CSRF)

8.1 Testing

CSRF was tested against a state-changing password operation while an authenticated DVWA session was active. A crafted request was used to trigger the password-change action without requiring the user to intentionally submit the form.

8.2 Result and Observation

The password-change operation was successfully triggered, demonstrating that the tested request did not adequately verify whether the action originated from an authorized application form. This confirmed the presence of a CSRF vulnerability in the tested functionality.

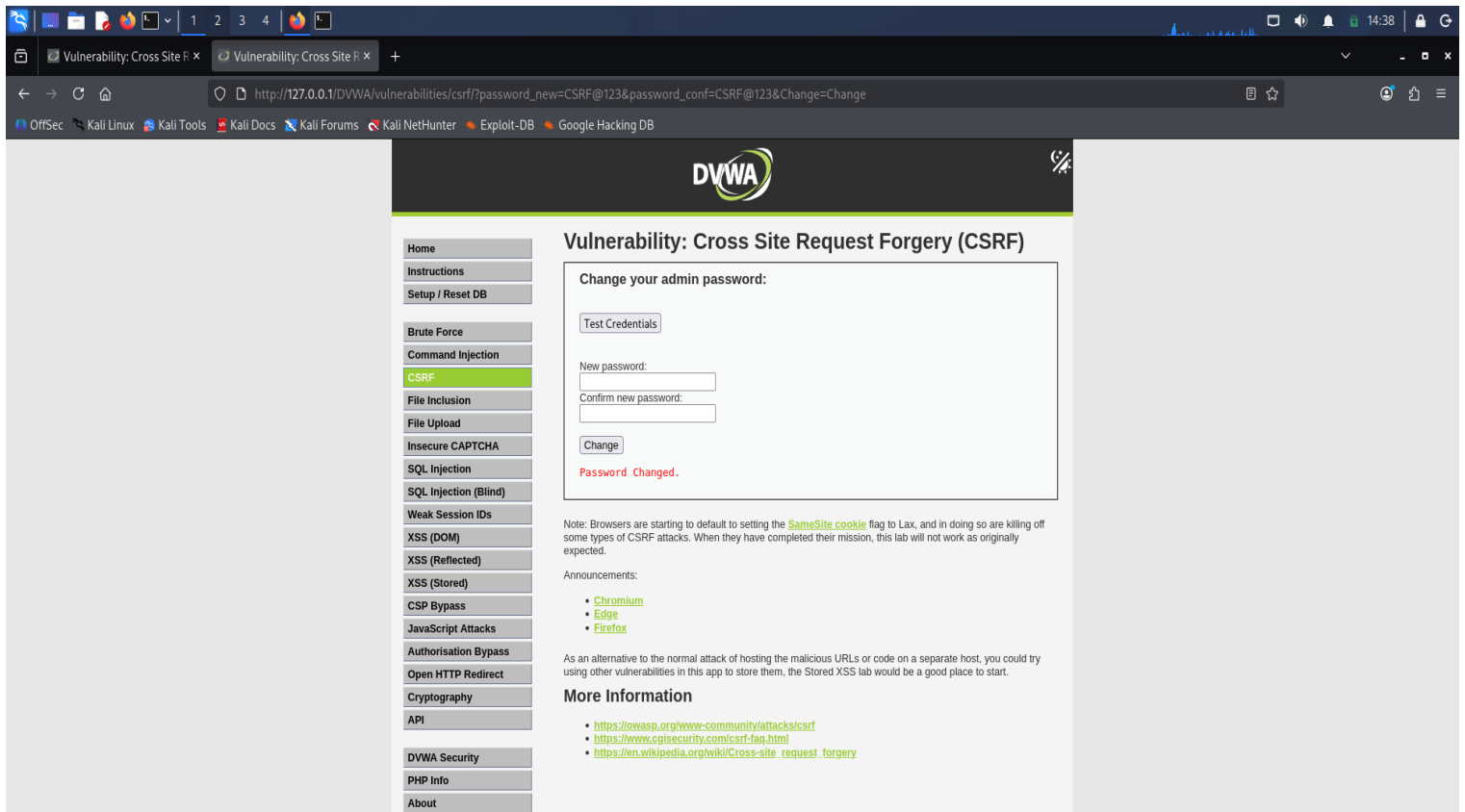


Figure 6: DVWA CSRF – Password Changed

8.3 Mitigation

State-changing requests should use unpredictable, session-bound CSRF tokens that are validated by the server.

Appropriate SameSite cookie settings should also be used as an additional protection, along with secure session management.

9. Security Findings

Finding	Severity	Evidence	Main Mitigation
SQL Injection	High	Multiple records and user/password hashes exposed	Parameterized queries and input validation
Stored XSS	Medium	Stored JavaScript executed in browser	Input validation, output encoding, CSP
Reflected XSS	Medium	Reflected JavaScript executed in browser	Input validation and output encoding
CSRF	Medium	Password-change request successfully triggered	CSRF tokens and cookie protections

10. Results

The assessment successfully demonstrated four web application security vulnerabilities in the controlled DVWA environment. SQL Injection allowed additional records and user credential hashes to be exposed. Stored and Reflected XSS allowed JavaScript to execute in the browser. CSRF

allowed a state-changing password operation to be triggered through a crafted request.

11. Security Observations

The testing showed the importance of validating and safely handling user-controlled input. SQL Injection can expose backend database information, while XSS can allow attacker-controlled scripts to execute in a user's browser. CSRF demonstrates the need to protect authenticated state-changing operations against unauthorized cross-site requests. These vulnerabilities can be significantly reduced through secure coding practices, proper input handling, output encoding, parameterized database queries, and strong request validation.

12. Conclusion

Task 3 successfully demonstrated practical web application security testing using DVWA. SQL Injection, Stored XSS, Reflected XSS, and CSRF were reproduced in a controlled local environment and documented with evidence. The assessment also identified appropriate mitigation measures for each vulnerability. The exercise provided practical understanding of how common web application vulnerabilities occur and how secure development practices can reduce their impact.